torch.fx.experimental.symbolic_shapes.c

https://pytorch.org/docs/stable/generated/torch.fx.experimental.symbolic_sha

Description:

Applies a constraint that the passed in SymInt must lie between min-max inclusive-inclusive, WITHOUT introducing a guard on the SymInt (meaning that it can be used on unbacked SymInts). If min/max are None, we assume that the dimension is unbounded in that direction. Repeated application of constrain_range intersects the ranges. This is a fairly low level API that doesn't have a lot of safety guarantees (TODO: provide higher level APIs). Currently, we use this API in the following circumstance: when we allocate an unbacked SymInt, denoting an integer quantity which is data dependent, we ordinarily do not know anything about what values it may take. This means that any sort of guard on it will immediately fail. However, in many cases, we know something about the unbacked SymInt: for example, we know that nonzero(x).size(0) must be >= 0. We use constrain_range to narrow the possible range, declaring that negative symbols are impossible. This permits to definitely answer True to queries like 'nnz >= 0', even if we don't know what the actual (hinted) value of 'nnz' is. In fact, we actually use constrain_range to unsoundly discharge common guards: for an unbacked SymInt produced

Function Signature

```
torch.fx.experimental.symbolic_shapes.constrain_range(a, *,
min, max=None)[source]#
```

Detailed Documentation

Documentation

Applies a constraint that the passed in SymInt must lie between min-max inclusive-inclusive, WITHOUT introducing a guard on the SymInt (meaning that it can be used on unbacked SymInts). If min/max are None, we assume that the dimension is unbounded in that direction. Repeated application of constrain_range intersects the ranges. This is a fairly low level API that doesn't have a lot of safety guarantees (TODO: provide higher level APIs).

Currently, we use this API in the following circumstance: when we allocate an unbacked SymInt, denoting an integer quantity which is data dependent, we ordinarily do not know anything about what values it may take. This means that any sort of guard on it will immediately fail. However, in many cases, we know something about the unbacked SymInt: for example, we know that nonzero(x).size(0) must be >= 0. We use constrain_range to narrow the possible range, declaring that negative symbols are impossible. This permits to definitely answer True to queries like 'nnz >= 0', even if we don't know what the actual (hinted) value of 'nnz' is. In fact, we actually use constrain_range to unsoundly discharge common guards: for an unbacked SymInt produced by nonzero, we will also assume that it is not equal to 0/1 (even though these are perfectly possible values at runtime), because we generally expect graphs that are valid for N=2 to also be valid for N=1.